# PulseEDU Migration Guide

Everything shipped July 1–2, 2026

For: development team | Scope: database schema, server enforcement, APIs, admin UI

*Earlier work (June 29–30: gradebook importer + GPA, student metrics engine, data export, pullout notifications) is covered by the prior guides in exports/. This document covers the July 1–2 window only.*

## 1. Shipped in this window (overview)

July 1:

- Delegable Data Importers — 4 assignable staff caps let a non-admin clerk run a single importer.
- Data Import UX overhaul — consolidated wizard, options grouped by usage, sample downloads.
- Eligibility Hub — activity management controls; team roster status reports (CSV + PDF); teacher/period filters on export.
- Header redesign — user controls consolidated into an account (kebab) menu, school-color themed.
- Historical FAST — multi-year FAST storage + multi-year table on Student Profile (admin-only cap, dev seed).
- FAST parity — Student Snapshot + Family HeartBEAT PDF now render the exact Teacher-Roster FAST presentation (single-sourced).
- Staff 'Print HeartBEAT' — staff can download the same family HeartBEAT PDF a parent gets, for data chats.
- HeartBEAT official absences — attendance metrics now come from the official upload and are omitted (never zeroed) when a source system is not in use.
- Demo data — seeded 2025–26 attendance + full grade report for School 1.

July 2:

- Data Chat Campaigns — complete admin-pushed teacher!”student check-in system (templates, campaigns, scopes, teacher queue, inline roster chat, history/compliance).
- Data Chat Follow-Ups — teachers schedule one pending follow-up per student, with roster reminders, snooze, and auto-complete on logging.
- Shared TeacherPicker — every teacher chooser standardized to one component (search + department-grouped select).
- Feature Enablement + Staff Pilots — 9 formerly always-on modules now individually licensable; per-staff pilot mode.

## 2. Database changes (all of them)

### 2.1 How migrations run

No drizzle-kit migration files. All DDL is idempotent boot-time SQL in artifacts/api-server/src/seed.ts (CREATE TABLE / ALTER TABLE ... IF NOT EXISTS). It runs automatically on every API-server start, so any environment self-migrates on first boot. The SQL below is the manual equivalent — safe to re-run.

### 2.2 NEW tables — Data Chats (5 tables)

```
CREATE TABLE IF NOT EXISTS data_chat_templates (
  id SERIAL PRIMARY KEY, school_id INTEGER NOT NULL,
  name TEXT NOT NULL, kind TEXT NOT NULL DEFAULT 'custom',
  built_in BOOLEAN NOT NULL DEFAULT FALSE,
  checklist_json TEXT NOT NULL DEFAULT '[]',
  goal_chips_json TEXT NOT NULL DEFAULT '[]',
  share_with_families BOOLEAN NOT NULL DEFAULT TRUE,
  archived BOOLEAN NOT NULL DEFAULT FALSE,
  created_by_staff_id INTEGER, created_at TIMESTAMPTZ, updated_at TIMESTAMPTZ);

CREATE TABLE IF NOT EXISTS data_chat_campaigns (
  id SERIAL PRIMARY KEY, school_id INTEGER NOT NULL, template_id INTEGER,
  name TEXT NOT NULL, kind TEXT NOT NULL, subject TEXT,      -- ela|math|both
  assignment_mode TEXT, selected_teacher_ids_json TEXT,
  responsible_period INTEGER, scope_json TEXT,
  checklist_json TEXT, goal_chips_json TEXT,                 -- snapshotted at launch
  share_with_families BOOLEAN NOT NULL DEFAULT TRUE,
  start_date TEXT, deadline TEXT, active BOOLEAN NOT NULL DEFAULT TRUE,
  ended_at TIMESTAMPTZ, created_by_staff_id INTEGER,
  created_by_name TEXT, created_at TIMESTAMPTZ);

CREATE TABLE IF NOT EXISTS data_chat_campaign_students (   -- narrowed-scope snapshot
  id SERIAL PRIMARY KEY, school_id INTEGER NOT NULL,
  campaign_id INTEGER NOT NULL, student_id TEXT NOT NULL,
  teacher_staff_id INTEGER NOT NULL, subject TEXT);
CREATE UNIQUE INDEX IF NOT EXISTS data_chat_campaign_students_pair_unique
  ON data_chat_campaign_students (campaign_id, teacher_staff_id, student_id);

CREATE TABLE IF NOT EXISTS data_chat_logs (
  id SERIAL PRIMARY KEY, school_id INTEGER NOT NULL,
  campaign_id INTEGER NOT NULL, student_id TEXT NOT NULL,
  teacher_staff_id INTEGER NOT NULL, entry_seq INTEGER NOT NULL DEFAULT 0,
  subject TEXT, discussed_json TEXT, goal TEXT,
  private_note TEXT,                      -- NEVER surfaces to parents
  created_at TIMESTAMPTZ, updated_at TIMESTAMPTZ);
CREATE UNIQUE INDEX IF NOT EXISTS data_chat_logs_pair_seq_unique
  ON data_chat_logs (campaign_id, teacher_staff_id, student_id, entry_seq);

CREATE TABLE IF NOT EXISTS data_chat_followups (
  id SERIAL PRIMARY KEY, school_id INTEGER NOT NULL,
  teacher_staff_id INTEGER NOT NULL, student_id TEXT NOT NULL,
  due_date TEXT NOT NULL,                 -- YYYY-MM-DD, weekends roll forward
  status TEXT NOT NULL,                   -- pending|done|cancelled
  snooze_count INTEGER NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ, updated_at TIMESTAMPTZ);
-- indexes: data_chat_followups_teacher_idx (school_id, teacher_staff_id, status)
--          data_chat_followups_student_idx (school_id, student_id)
```

*Drizzle schema: lib/db/src/schema/dataChats.ts. One pending follow-up per teacher+student is enforced at the API layer.*

### 2.3 NEW table — staff_feature_pilots

```
CREATE TABLE IF NOT EXISTS staff_feature_pilots (
  id SERIAL PRIMARY KEY, school_id INTEGER NOT NULL,
  feature_key TEXT NOT NULL,               -- validated against FEATURE_KEYS
  staff_id INTEGER NOT NULL, granted_by_staff_id INTEGER,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW());
CREATE UNIQUE INDEX IF NOT EXISTS staff_feature_pilots_unique
  ON staff_feature_pilots (school_id, feature_key, staff_id);
```

### 2.4 staff table — 5 new capability columns (all DEFAULT FALSE)

```
ALTER TABLE staff ADD COLUMN IF NOT EXISTS cap_import_grades     BOOLEAN NOT NULL DEFAULT FALSE;
ALTER TABLE staff ADD COLUMN IF NOT EXISTS cap_import_attendance BOOLEAN NOT NULL DEFAULT FALSE;
ALTER TABLE staff ADD COLUMN IF NOT EXISTS cap_import_fast       BOOLEAN NOT NULL DEFAULT FALSE;
ALTER TABLE staff ADD COLUMN IF NOT EXISTS cap_import_iready     BOOLEAN NOT NULL DEFAULT FALSE;
ALTER TABLE staff ADD COLUMN IF NOT EXISTS cap_view_fast_history BOOLEAN NOT NULL DEFAULT FALSE;
```

*First four = delegable importers (grades / attendance / FAST / iReady). Fifth = historical FAST visibility.*

### 2.5 student_fast_scores — multi-year FAST

```
ALTER TABLE student_fast_scores ADD COLUMN IF NOT EXISTS school_year TEXT NOT NULL;  -- e.g. '25-26'
ALTER TABLE student_fast_scores ADD COLUMN IF NOT EXISTS is_historical BOOLEAN NOT NULL DEFAULT FALSE;
ALTER TABLE student_fast_scores ADD COLUMN IF NOT EXISTS imported_as_historical_at TIMESTAMPTZ;
-- unique index REPLACED:
--   old: (school_id, student_id, subject)
--   new: student_fast_scores_student_subject_year_unique
--        (school_id, student_id, subject, school_year)
```

*IMPORTANT: every query on student_fast_scores that expects 'current year' rows must now filter by school_year — prior-year Florida backfill rows share the table.*

### 2.6 school_settings — 18 new license columns (all DEFAULT TRUE)

```
-- one pair per module; <col> in: data_chats, pickup, ticketing, tours,
--   esign, brain_lab, gradebook, school_grade, safety_plans
ALTER TABLE school_settings
  ADD COLUMN IF NOT EXISTS feature_<col>       BOOLEAN NOT NULL DEFAULT TRUE;
ALTER TABLE school_settings
  ADD COLUMN IF NOT EXISTS super_feature_<col> BOOLEAN NOT NULL DEFAULT TRUE;
```

### 2.7 plans JSONB backfill (only-if-absent)

Every plan's features JSONB gets the 9 new keys set true ONLY if absent — otherwise the next plan re-apply would flip the formerly-always-on modules OFF for every school on a plan:

```
UPDATE plans SET features = features || jsonb_build_object(
  'dataChats',   COALESCE(features->'dataChats',   'true'::jsonb),
  'pickup',      COALESCE(features->'pickup',      'true'::jsonb),
  'ticketing',   COALESCE(features->'ticketing',   'true'::jsonb),
  'tours',       COALESCE(features->'tours',       'true'::jsonb),
  'esign',       COALESCE(features->'esign',       'true'::jsonb),
  'brainLab',    COALESCE(features->'brainLab',    'true'::jsonb),
  'gradebook',   COALESCE(features->'gradebook',   'true'::jsonb),
  'schoolGrade', COALESCE(features->'schoolGrade', 'true'::jsonb),
  'safetyPlans', COALESCE(features->'safetyPlans', 'true'::jsonb));
```

### 2.8 Demo/seed data (dev only)

- Seeded 2025–26 attendance rows (student_attendance_day) for demo schools.
- Full FAKE grade report xlsx for School 1 (exports/FAKE_Student_LIVE_Grade_Report-School1-Full.xlsx) + populated teacher names — for gradebook-importer demos.
- Historical FAST dev seed for the multi-year table.

## 3. Data Chat Campaigns (July 2) — how it works

### 3.1 Model

- Templates vs Campaigns: checklist / goal chips / share-with-families are SNAPSHOTTED into the campaign at launch — later template edits do not change a running campaign.
- Built-in 'FAST Data Chat' template is lazily seeded per school; name/kind locked; DELETE archives.
- FAST campaigns assign students to their ELA/Math teacher-of-record (subject ela|math|both), OR with assignment:'selected' to hand-picked teachers of ANY department + a responsible period (workload equity). kind stays fast_data either way.
- Student scope at launch: All / support flags (ese, is504, ell) / D-or-F in class (grade < 70) / hand-picked (max 500). Narrow scopes are snapshotted into data_chat_campaign_students and applied via resolvePairs() everywhere downstream.
- Teacher flow: animated top-bar reminder + deadline banner !’ queue modal (checklist, goal chips, FAST mini pills, past goals, private teacher note).
- Inline roster chat: Teacher Roster rows show a Chat button !’ SelfDataChatModal. Pending-campaign students log toward the campaign; others log to a hidden per-school self_serve campaign (excluded from queues/ reminders/lists).
- Parents see topics + goal on HeartBEAT only when the campaign shares; the private teacher note NEVER reaches parent payloads.

### 3.2 Key endpoints (routes/dataChats.ts)

```
GET/POST  /data-chats/templates          GET/POST /data-chats/campaigns
POST      /data-chats/campaigns/preview   (scope preview count)
GET       /data-chats/my-queue            POST /data-chats/logs
GET       /data-chats/pending-students    POST /data-chats/self-log
GET       /data-chats/followups/mine      (60s roster poll)
POST/PATCH followup schedule / snooze / cancel; GET /followups/admin-stats
```

### 3.3 Security invariants

- The log POST re-derives queue membership server-side (resolvePairs) — a teacher can only log their own assigned students.
- self-context / self-log require teacherHasStudent (own roster only).
- loadStaff 403s a non-SuperUser actor whose school differs from req.schoolId.

### 3.4 Follow-ups behavior

- ONE pending follow-up per teacher+student; reschedule replaces in place and resets snooze; weekend dates roll to the next school day.
- Roster reminders: quiet the school day before + due-day pre-period; LOUD from the earliest non-planning period with that student; overdue always loud. Snooze 1 or 3 school days; 'snoozed 3x' nudge.
- Logging ANY chat auto-completes the pending follow-up EXCEPT a row scheduled today for a future date

(discriminated by snooze_count = 0).

- Follow-ups never touch HeartBEAT / parent portal / support records / exports.

## 4. Feature Enablement + Staff Pilots (July 2)

Nine formerly always-on modules are now licensable: Data Chats, Pick-Up, Ticketing, Tours, E-Sign, Brain Lab, Gradebook, School Grade, Safety Plans. Effective enablement is computed in exactly one place (lib/featureLicensing.ts):

```
enabled = superOn && (schoolOn || actorHasPilotRow)
```

- District super switch always wins; school admins toggle via Settings > School Features; pilots let selected staff use a feature while the school toggle stays off.
- Family-facing keys (ticketing, parentPortal, familyComm, schoolStoreNotify) are pilotable:false — the pilot PUT 400s them.
- requireFeature route mounts return 404 when off (API stays dark). Public surfaces stay open: /ticketing/scan, /esign/sign/:token, /tours/public/*, /tours/walk/*. Parent routes use requireFeatureForParent.
- PUT /api/school-settings accepts the 9 camelCase keys (featureDataChats ... featureSafetyPlans).
- Pilot APIs: GET /api/school-features/pilots; PUT /api/school-features/pilots/:key {staffIds} — full-replace tx, active same-school staff only.
- Gradebook goes dark END-TO-END when off: importer preview/commit, rollback (404s gradebook jobs), job listing (404 ?kind=gradebook + omits rows), and embedded grade reads (loadCurrentGrades returns nothing unless BOTH school-level switches are on). Pilots re-enable the importer flow but NOT embedded reads — the shared loader has no staff context and feeds parent surfaces.
- Admin UI: School Features grid +9 rows; new FeaturePilotsPanel (per-feature staff picker).

*Every switch ships TRUE — zero behavior change until someone flips a toggle.*

## 5. Delegable Data Importers (July 1)

- Admin OR Core Team can hand a single importer to a non-admin clerk via 4 staff caps (cap_import_grades / attendance / fast / iready).
- Assigned via StaffRolesMatrix 'import-only mode' — Core Team without full role authority sees ONLY the 4 import toggles.
- A non-admin holder gets a dedicated Data Imports page (allowed kinds only) + the Eligibility upload tab for attendance.
- SERVER-side enforcement: canImportKind / allowedSchoolImportKinds gate every dataImports.ts surface (preview/commit/jobs/export/rollback/templates); attendance gated by requireAttendanceImporter in eligibility.ts.
- Anti-escalation: adminStaff.ts PATCH strips a non-full-authority Core Team actor's edits to ONLY the 4 import caps.

*Client filtering (allowedKinds, import-only matrix) is UX only and bypassable — the server gates are authoritative.*

## 6. Historical FAST + FAST parity + HeartBEAT (July 1)

### 6.1 Historical FAST

- student_fast_scores is now keyed by (school_id, student_id, subject, school_year); prior-year rows carry is_historical.
- Student Profile gained an admin/Core-Team multi-year FAST table (gated by cap_view_fast_history).
- Learning-gain checks source prior-year evidence from loadFastHistory historical PM3 — never the legacy

priorYearScore column.

### 6.2 FAST parity (single-source)

- Student Snapshot 'FAST Progress Monitoring' + Family HeartBEAT PDF now render Teacher-Roster-parity FAST: level pills/sub-levels, points-to-next-level, points-to-proficiency, learning-gain check.
- Single-sourced via lib/fastParity.ts loadStudentFastParity (placePmSet / bucketFor / proficiencyGap / computeRowLearningGain). If you change FAST presentation, change it THERE — all surfaces must agree.

### 6.3 Staff 'Print HeartBEAT'

- Button next to 'Preview as parent' on Parent Access (Admin/Core Team) downloads the SAME family HeartBEAT PDF a parent gets.
- GET /api/staff/heartbeat.pdf?studentId=<numeric DB id> (routes/staffHeartbeatPdf.ts) — staff-authed, visibility-scoped via getVisibleStudentIds, school-scoped, no parent account involved.
- Invariant: out-of-scope AND non-existent students return an indistinguishable 404 (no existence probing).

### 6.4 Official absences / omit-when-unused

- HeartBEAT surfaces show OFFICIAL days-absent (latest Eligibility Hub upload via lib/attendanceMetrics.ts) — the kiosk-derived absence estimate + disclaimer were removed.
- Every attendance metric OMITS when its source system is not in use: no hub upload !' no absence tile (null, never 0); no attendance-day rows !' % rows omitted; streak/arrival tiles gated on their own counts.
- Invariant: student_attendance_day remains the sole attendance-% source; the hub's approximate % is never shown.

## 7. Eligibility + UI polish (July 1)

- Eligibility Hub: managers can add/edit/archive activities in-app (management controls).
- Team roster status reports: GET /eligibility/activities/:id/roster.csv and .../roster.pdf — coach/AD-friendly eligibility status downloads.
- Data export gained optional teacher + period filters.
- Header: user controls consolidated into an account (kebab) menu, themed to the school's primary color; responsive fixes.
- Sidebar: re-clicking a nav item resets its section state.
- Data Import wizard: options organized by usage frequency, consolidated steps, sample-file downloads, clearer accessibility.

## 8. Invariants your team must preserve

- Feature enablement math lives ONLY in loadEffectiveFeatures(); a licensed module must go dark on EVERY surface together (mounts, rollback-style side doors, listings, embedded reads).
- Data-chat authorization is server-derived (resolvePairs / teacherHasStudent); private teacher notes never reach parent payloads.
- Importer authorization is enforced at each server route (canImportKind / requireAttendanceImporter); the adminStaff PATCH field-strip is the anti-escalation guard.
- student_fast_scores queries must filter by school_year for current-year semantics.
- FAST presentation is single-sourced (fastParity / FastScorePill / placePmSet) — never fork it per surface.
- Attendance metrics omit (null) when their source system is unused — never show 0 or an estimate.
- FLEID boundary: only local_sis_id is ever user-visible; students.student_id is internal.

## 9. File map (July 1–2)

```
DB schema (lib/db/src/schema/)
  dataChats.ts               NEW — 5 data-chat tables
  staffFeaturePilots.ts      NEW — pilot grants
  schoolSettings.ts          +18 license columns
  staff.ts                   +5 cap columns
  studentFastScores.ts       school_year / is_historical + new unique index
Server (artifacts/api-server/src/)
  seed.ts                    all idempotent DDL + plans backfill + dev seeds
  routes/dataChats.ts        NEW — campaigns/queue/logs/follow-ups
  routes/featureLicensing.ts pilots GET/PUT
  lib/featureLicensing.ts    +9 FEATURE_KEYS specs, pilot union
  routes/index.ts            requireFeature mounts
  routes/schoolSettings.ts   +9 PUT keys
  routes/dataImports.ts      importer caps + gradebook dark gates
  routes/eligibility.ts      roster.csv/.pdf, attendance importer gate
  routes/staffHeartbeatPdf.ts NEW — staff HeartBEAT PDF
  lib/fastParity.ts          NEW — single-source FAST presentation
  lib/attendanceMetrics.ts   official absences helper
  lib/studentMetrics.ts      gradebook read gate
Client (artifacts/client/src/)
  App.tsx                    nav/tile/section gating; header account menu
  components/FeaturePilotsPanel.tsx  NEW
  components/TeacherPicker.tsx       shared teacher chooser (standardized)
  (Data Chats UI: launcher, queue modal, SelfDataChatModal,
   FollowupReminders, history/compliance views)
Docs
  docs/shipped.md            detailed entries for every item above
  replit.md                  product bullets + invariants
```

## 10. Deploying this

- No manual DB step: seed.ts DDL runs on API-server boot and is idempotent — publishing migrates production automatically on first start.
- Managing a database by hand? Run the SQL in section 2 top-to-bottom (safe to re-run).
- Rollout risk: near zero. License switches ship TRUE (no behavior change); importer caps ship FALSE (nobody gains access); data chats are inert until an admin launches a campaign.
- Verified: full typecheck passes; curl E2E covered feature toggles, pilots, district precedence, importer caps, data-chat authorization, and gradebook dark-mode surfaces.